

ELI — Productization Path for a Solo Maintainer

Contents

0. The thesis (read this first)	1
1. The Core (the ~20% that must be flawless) — the “trust spine”	1
2. The Advanced/Experimental wall (the ~80% to gate)	2
3. GUI productization (Simple Mode default)	3
4. The liability + distribution reality (solo-specific, be clear-eyed)	3
5. Metrics that actually matter (not vanity)	3
6. Phased roadmap (gated, solo-realistic)	4
7. Non-goals (explicit — protect focus)	4
8. One-paragraph summary	4

Purpose: A strategy reference for turning ELI from an impressive system into something a stranger can trust on day one — without a team. It answers one question: *which ~20% of the surface must be flawless, and which ~80% should be gated behind “Advanced/Experimental”?*

Companion docs: `project_audit.md` (current state + known-issues register), `adaptive_inference_governor.r` (the latency/throughput fix that this plan depends on).

Snapshot: 2026-06-17, commit a641471.

0. The thesis (read this first)

The gap between ELI and a commercial product is **not features** — ELI already has *more* capability surface than most things in its lane. The gap is **reliability, latency, sane defaults, and trust**. Therefore productization is not “build more.” It is **triage**:

Narrow the promise to a small Core that is *flawless and fast*; gate everything else behind an explicit Advanced/Experimental wall so it can’t break the first-run experience.

A new user must never meet a 9-minute turn, a confabulated answer, a faked action, or a shell-exec footgun in the first ten minutes. Everything that risks that goes behind a flag.

Positioning (the honest lane): *“The deepest private, fully-local, autonomous AI you can actually own.”* Not a frontier-assistant competitor. Privacy-absolute + real agency is the wedge; lean into it, don’t apologize for the model being local.

1. The Core (the ~20% that must be flawless) — the “trust spine”

These are the things a new user will touch immediately and judge on. They must be **fast, deterministic, and honest** — and they’re mostly the actions whose executor result is already authoritative (`_PHASE45_DIRECT_FAST_ACTIONS`, `engine.py:78`).

Core surface	Why it's Core	Bar
Chat (quick mode)	the front door	responsive (streamed), never empty, never confabulates internals
Deterministic OS actions — screenshot, open/close app, media play/pause/volume, file create/list, notes	"does what it says" is the whole trust test	execute really happen; reply = executor result, never narrated success
Job/status & background tasks	users wait on these	honest status, real registry (CHECK_JOB)
Grounded introspection — "what can you do / what's in memory / status"	your unique selling point; must not lie	disk-probed, never CHAT-guessed
Privacy/offline guarantee	the headline promise	verifiable: nothing leaves the box; netguard on by default
First-run hardware autodetect	decides whether ELI feels broken	picks a model + ctx the machine can actually serve at an interactive speed
Crisis/safety guard	non-negotiable	always on, never gated

Hardening contract for every Core action (definition of "done"): 1. **Golden-path test** — the happy case asserted in tests/ (you already have 7,347 passing tests; add an explicit "Core golden path" suite). 2. **Router fuzz** — paraphrase/typo/STT-variant inputs route correctly (the router is the #1 bug surface; tests/router_test_data.json is the seed). 3. **Latency budget** — projected wall-time $\leq$ target (depends on the governor); if it can't meet it, it streams or shows progress, never silently hangs. 4. **Honest failure** — on failure it says "I couldn't do X because Y," never fakes success and never confabulates (No-Fake-Actions is already a stated invariant — make it a *tested* one for Core).

2. The Advanced/Experimental wall (the ~80% to gate)

All of this is genuinely valuable and is what makes ELI special — but it is *powerful, slow, or unreliable* and must not be in the first-run path. Gate behind a single explicit "Advanced features (experimental)" reveal, **off by default**, with a one-line risk note each.

Surface	Why gated	Default
Full Control / SHELL_EXEC (the red button)	runs arbitrary shell on the user's machine — the single biggest liability	OFF , explicit opt-in + warning, fail-closed
Think mode (Think: ON today)	the empty-<think>/latency footgun on slow HW	governor-driven (auto), manual override is Advanced
Proactive autonomy / self-improvement / autonomy tick	acts/learns without the user; churn + surprise	OFF or low-key; opt-in
Coding agent (CODE_SOLVE)	powerful but unreliable; long runs	Advanced tab only
Ambient vision (Watch), Gaze	privacy-sensitive, always-on capture	OFF , explicit consent
Eli's World / avatar rooms	flavour, not function; leaked into chat (audit I10)	Advanced/optional
Exotic reasoning strategies (ToT, constitutional, self-consistency, research/expert modes)	multiply latency; quick/normal cover 95%	quick/normal Core; rest Advanced

Surface	Why gated	Default
Image diffusion (SSD-1B), LoRA training, eval harness, Report Builder	dev/power-user tools	Advanced/Labs
Network features (news, web)	breaks the offline promise if default-on	opt-in, clearly “this goes online”

Principle: **a feature is Core only if a new user benefits from it in the first session AND it can meet the Core bar today.** Everything else waits behind the wall — visibly present (so power users find it), but never load-bearing for first impressions.

3. GUI productization (Simple Mode default)

The current GUI is a 14-tab, ~14-toggle cockpit — perfect for *you*, overwhelming and footgun-laden for a new user (e.g. Full Control, Think: ON, Gaze, Watch exposed as front-line buttons).

- **Default = Simple Mode:** Chat + send/clear + mic + a *privacy/offline* indicator + a model status chip. Nothing that can break trust on screen.
- **One “Advanced” toggle** reveals the tabs (Coding, Labs, Tasks, Orchestration, Eli’s World, Report Builder, Test Review) and the deep toggles.
- **Smart defaults do the work** the toggles currently expose: the governor picks Think on/off and budget from hardware; netguard stays on; Full Control stays off. The user shouldn’t have to *know* what Think does.
- **Model chip honesty:** show a friendly name + “(local, private)”; don’t surface a confident non-canonical filename (Qwen3.6-35B-A3B) as if authoritative to a layperson.

4. The liability + distribution reality (solo-specific, be clear-eyed)

- **Full Control is the crux.** A one-person-maintained agent that can run shell commands and drive the machine is a hard *mainstream* sell and a real liability. Keep it: (a) off by default, (b) fail-closed via the existing SecurityManager/approval_engine, (c) behind an explicit consent screen, (d) clearly “expert/at-your-own-risk.” Don’t make it the headline.
- **Support doesn’t scale to one person.** Favor a model that minimizes inbound support: excellent first-run autodetect, clear in-app diagnostics (you already have introspection — expose a “self-diagnose” the user can copy-paste), and docs over hand-holding.
- **Cross-platform is a tax.** The Linux/macOS/Windows mandate is real work; treat Windows as a first-class target only when the Core spine passes on it (CI matrix, not manual).
- **Telemetry-free is an asset, not a gap.** Lean into “we literally can’t see your data” — but that means you get no crash telemetry, so the in-app self-diagnose + opt-in bug export matters.
- **Licensing/redistribution:** the no-personal-data + portable-settings work is the right track; finish stripping machine/user-specific values before any public build.

5. Metrics that actually matter (not vanity)

Define “shippable” by these, measured on *varied* hardware, not your dev box: 1. **Cold-start → first useful turn** (seconds). The single biggest first-impression number. 2. **% of turns under the latency target** (governor target). 3. **“Did-what-it-said” rate** for Core actions (executed == claimed). Target ~100%. 4. **Confabulation rate** on grounded/introspection questions. Target ~0. 5. **Crash-free session rate.** 6. **First-run success without docs** (a stranger gets value without reading anything).

If these are green on a mid-range laptop and a no-GPU box, ELI is a product. Feature count is irrelevant to this list — which is the whole point of the triage.

6. Phased roadmap (gated, solo-realistic)

Each phase has an exit gate; don't advance until it's green.

Phase A — Core hardening (you). - Land the **Adaptive Inference Governor** (latency is gate-zero). - Lock the Core action set to No-Fake-Actions + honest-failure, with golden-path + router-fuzz tests. - First-run autodetect that picks a servable model/ctx for the detected hardware. - *Gate*: every Core metric green on your box AND one weaker box (or simulated slow tok/s).

Phase B — Simple Mode + the Advanced wall. - Default Simple Mode GUI; move the 80% behind the Advanced reveal; Full Control/Think/ambient default-off. - In-app self-diagnose (reuse introspection) + opt-in bug export. - *Gate*: a non-you human completes a useful session on a fresh machine with no instructions.

Phase C — Private beta (3-10 people, varied hardware). - Real machines you don't control. Watch the six metrics. Fix only Core regressions; log the rest behind the wall. - *Gate*: crash-free + did-what-it-said $\approx 100\%$ across the cohort; cold-start acceptable on the weakest machine.

Phase D — Public beta (gated) → 1.0. - Public build with Advanced features clearly "experimental." 1.0 = the six metrics hold on the hardware floor you commit to support, and Full Control's consent/audit path is solid.

7. Non-goals (explicit — protect focus)

- **Do not** chase frontier-model parity. You will lose and it's the wrong lane.
 - **Do not** add features to reach "product." You have more than enough; harden, don't expand.
 - **Do not** ship Full Control, ambient vision, or Think-on-by-default in the first-run path.
 - **Do not** let the offline/privacy promise be quietly broken by a default-on network feature.
 - **Do not** treat the dev box as representative — every gate is measured on weaker hardware.
-

8. One-paragraph summary

ELI's path to "commercial-grade" is **subtraction, not addition**: define a small Core (responsive chat, a handful of deterministic do-what-they-say actions, honest grounded introspection, an ironclad privacy/offline promise, and hardware-aware first-run), make that Core flawless and fast via the governor + No-Fake-Actions tests, and put the entire powerful- but-risky remainder (Full Control, autonomy, coding agent, ambient vision, exotic reasoning, world avatar, Labs) behind an explicit Advanced/Experimental wall that is off by default. Judge "shippable" by six hardware-honest metrics, not feature count. That turns the most ambitious solo AI system in its lane into something a stranger can trust in ten minutes — which is the only thing standing between "remarkable project" and "product."